

DDD数智通 · 使用统计功能说明

一、整体原理

统计了什么？

事件	触发时机
page_view 网页打开	每次用户打开网页时自动上报
pwa_install PWA安装	用户将网页"安装到桌面"时上报
calc_click 计算DDDs	用户点击"☑ 计算DDDs及使用强度"按钮时上报

双环境自动路由

网页加载时，统计模块会检查当前访问地址，**自动判断是公网还是内网**，选择对应的上报渠道：



两套渠道**完全独立**，互不干扰。任何一侧的故障（网络不通、服务未启动等）都会静默失败，**不影响主功能的正常使用**。

二、公网发布

1、Cloudflare Workers 源码 (无需外接任何数据库)

请在 Cloudflare Workers 控制台创建一个新项目，并将以下代码粘贴到 `index.js` 或 `worker.ts` 中。

JavaScript

```
1 export default {
2   async fetch(request, env, ctx) {
3     // 处理跨域请求 (CORS)
4     const corsHeaders = {
5       "Access-Control-Allow-Origin": "*",
6       "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
7       "Access-Control-Allow-Headers": "Content-Type",
8     };
9
10    if (request.method === "OPTIONS") {
11      return new Response(null, { headers: corsHeaders });
12    }
13
14    const url = new URL(request.url);
15
16    // 检查 KV 绑定是否存在
17    if (!env.DDD_STATS_KV) {
18      return new Response(JSON.stringify({ error: "KV 命名空间
19      'DDD_STATS_KV' 未绑定! " }), {
20        status: 500,
21        headers: { ...corsHeaders, "Content-Type": "application/json" }
22      });
23
24    // == 路由1: 上报统计数据 (POST /track) ==
25    if (url.pathname === "/track" && request.method === "POST") {
26      try {
27        const body = await request.json();
28        const { event, ua } = body;
29
30        const validEvents = ["page_view", "pwa_install", "calc_click"];
31        if (!validEvents.includes(event)) {
32          return new Response(JSON.stringify({ error: "无效的事件名称" }), {
33            status: 400, headers: corsHeaders });
34        }
35
36        // 1. 原子自增总计数
37        // 由于 workers 默认没有直接自增方法，我们在 KV 中读取后加 1 再写回
38        const countKey = `count:${event}`;
39        const currentCount = parseInt(await env.DDD_STATS_KV.get(countKey)
40        || "0");
41        await env.DDD_STATS_KV.put(countKey, (currentCount +
42        1).toString());
43
44        // 2. 插入最新日志 (保存最近20条)
45        const logListKey = "stats_logs";
46        const logsRaw = await env.DDD_STATS_KV.get(logListKey) || "[]";
47        let logs = JSON.parse(logsRaw);
48
49        // 获取北京时间字符串
```

```

47     const bjTime = new Date(new Date().getTime() + 8 *
3600000).toISOString()
48         .replace("T", " ")
49         .substring(0, 19);
50
51     const eventLabels = {
52         "page_view": "网页打开",
53         "pwa_install": "PWA 安装",
54         "calc_click": "计算DDDs"
55     };
56
57     const newLog = {
58         event: event,
59         label: eventLabels[event] || event,
60         time: bjTime,
61         ua: ua || "Unknown"
62     };
63
64     logs.unshift(newLog); // 插入到数组开头
65     logs = logs.slice(0, 20); // 仅保留最近20条
66
67     await env.DDD_STATS_KV.put(logListKey, JSON.stringify(logs));
68
69     return new Response(JSON.stringify({ success: true }), {
70         headers: { ...corsHeaders, "Content-Type": "application/json" }
71     });
72 } catch (err) {
73     return new Response(JSON.stringify({ error: err.message }), {
74         status: 500, headers: corsHeaders });
75 }
76
77 // == 路由2: 获取统计结果 (GET /api/stats) ==
78 if (url.pathname === "/api/stats" && request.method === "GET") {
79     try {
80         const pv = parseInt(await env.DDD_STATS_KV.get("count:page_view")
81         || "0");
82         const inst = parseInt(await
83         env.DDD_STATS_KV.get("count:pwa_install") || "0");
84         const calc = parseInt(await
85         env.DDD_STATS_KV.get("count:calc_click") || "0");
86
87         const logsRaw = await env.DDD_STATS_KV.get("stats_logs") || "[]";
88         const logs = JSON.parse(logsRaw);
89
90         const result = {
91             counts: {
92                 page_view: pv,
93                 pwa_install: inst,
94                 calc_click: calc
95             },
96             logs: logs
97         };
98
99         return new Response(JSON.stringify(result), {
100             headers: { ...corsHeaders, "Content-Type": "application/json" }
101         });
102     } catch (err) {

```

```

100         return new Response(JSON.stringify({ error: err.message }), {
101             status: 500, headers: corsHeaders });
102     }
103
104     // 404 页面
105     return new Response("Not Found", { status: 404, headers: corsHeaders
106 });
107 }
108 };

```

2、Cloudflare 部署步骤（新手几分钟即可搞定）

1. 登录到 [Cloudflare Dashboard](#)。
2. 在左侧菜单中点击 **Workers & Pages (Workers 和页面)** -> 点击 **Create (创建)** -> **Create Worker (创建 Worker)**。
3. 给项目起个名字（例如 `ddd-stats-api`），点击 **Deploy (部署)**。
4. 部署后点击 **Edit Code (编辑代码)**，将上方的 JavaScript 源码全部替换进去，然后点击右上角的 **Save and Deploy (保存并部署)**。
5. **绑定 KV 存储（关键步骤）：**
 - 第一步：新建一个 KV 空间（Namespace）
 1. **在新标签页中打开 Cloudflare**（或者直接点击左侧最外层的返回箭头，回到 Cloudflare 的大首页）。
 2. 在 Cloudflare 左侧最外层的主菜单中，找到 **Storage & Databases (存储与数据库)** -> 点击 **KV**。
 3. 进入 KV 页面后，点击右上角的 **Create a namespace (创建命名空间)** 按钮。
 4. 在弹出的输入框中输入名字：`ddd_kv_data`
 5. 点击 **Add (添加/保存)**。
 - 第二步：回到当前的 Worker 页面进行绑定
 1. 完成第一步后，切回 Worker 变量配置页面。
 2. **刷新一下网页**（让网页刷新读取到你刚刚新建的 KV 空间）。
 3. 重新点击 **Add binding (添加绑定)**：
 - **Variable name (变量名称)** 照常输入：`DDD_STATS_KV`（注意必须全大写）
 - **KV namespace (KV 命名空间)**：再次点击这个下拉框，你就会发现刚刚创建的 `ddd_kv_data` 已经躺在里面了，直接选中它。
 4. 点击最下方的 **Save (保存)** 按钮。
6. 回到 Worker 的 **Summary (概述)** 页，你会看到一个类似 `https://ddd-stats-api.<你的用户名>.workers.dev` 的 **Routes (路由)** 地址。**复制这个地址**，这就是你的统计后端！
目前创建的是：<https://ddd-stats-api.adamhtmei.workers.dev>

三、内网发布

部署统计服务

将 `stats_server.py` 放到内网服务器任意目录，执行：

```
1 | python3 stats_server.py
```

无需安装任何第三方库，**Python 3.6+ 即可运行**。

启动成功后终端输出:

```
1 | ┌───────────────────────────────────────────────────────────┐
2 | │ DDD数智通 统计服务已启动                                │
3 | │ ────────────────────────────────────────────────────────────┘
4 | │ 接收地址: http://0.0.0.0:8765/track                      │
5 | │ 查看报告: http://localhost:8765/report                    │
6 | │ 数据文件: stats_data.json                                  │
7 | └───────────────────────────────────────────────────────────┘
```

填写配置

在 DDD_system.html 的 CONFIG 区域, 填写内网服务器地址:

```
1 const INTRANET_API = 'http://192.168.1.100:8765/track'; // 替换为实际内网IP
2
3 // 内网环境判断特征（默认已覆盖常见内网IP段，一般无需修改）
4 const INTRANET_HINTS = ['192.168.', '10.', '172.16.', 'localhost', '127.0.'];
```

如果内网使用域名访问（如 `ddd.hospital.local`），在 `INTRANET_HINTS` 中添加该域名关键词即可：

```
1 const INTRANET_HINTS = ['192.168.', '10.', '172.16.', 'hospital.local'];
```

查看内网统计结果

浏览器打开统计看板：

1 | <http://内网服务器IP:8765/report>

页面显示三项累计数字 + 最近20条访问记录:

1			
2	128	17	89
3	网页打开次数	PWA安装次数	计算DDDS次数
4			
5			
6	时间	事件	客户端
7	2025-06-10 14:32	计算DDDS及使用强度	Mozilla/5.0 (windows...
8	2025-06-10 14:31	网页打开	Mozilla/5.0 (iPhone...
9	...		

或直接查看 JSON 原始数据:

1 <http://内网服务器IP:8765/api/stats>

返回：

```
1 {
2   "counts": {
3     "page_view": 128,
4     "pwa_install": 17,
5     "calc_click": 89
6   },
7   "logs": [...]
8 }
```

或直接查看数据文件：

```
1 cat stats_data.json
```

设置开机自启（Linux 推荐）

```
1 sudo nano /etc/systemd/system/ddd-stats.service
2 [Unit]
3 Description=DDD数智通统计服务
4 After=network.target
5
6 [Service]
7 ExecStart=/usr/bin/python3 /your/path/stats_server.py
8 WorkingDirectory=/your/path/
9 Restart=always
10 RestartSec=5
11
12 [Install]
13 WantedBy=multi-user.target
14 sudo systemctl daemon-reload
15 sudo systemctl enable ddd-stats      # 开机自启
16 sudo systemctl start ddd-stats      # 立即启动
17 sudo systemctl status ddd-stats     # 查看状态
```

四、文件清单

文件	作用	部署位置
DDD_system.html	主网页，含统计上报逻辑	GitHub 仓库 / 内网 Web 服务器
stats_server.py	内网统计接收服务	内网服务器（仅内网需要）
stats_data.json	内网统计数据文件（自动生成）	与 stats_server.py 同目录
manifest.json	PWA 配置	与 DDD_system.html 同目录
sw.js	Service Worker（PWA 离线缓存）	与 DDD_system.html 同目录
icons (图标)	各平台显示的图标	与 DDD_system.html 同目录

五、常见问题

Q：公网和内网的数据是分开统计的吗？ 是的，完全独立。公网数据在 Cloudflare的KV，内网数据在 `stats_data.json`，不会合并。

Q：统计失败会影响主功能吗？ 不会。所有上报请求都是异步发出、静默失败（`.catch(() => {})`），即使服务器宕机或网络不通，网页功能完全不受影响。

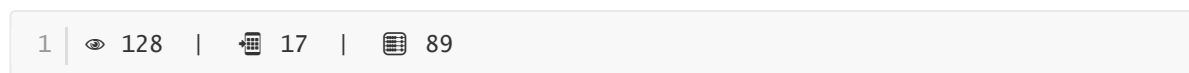
Q：内网 `stats_server.py` 没启动时，内网用户访问会报错吗？ 不会。fetch 请求失败会被静默捕获，用户不会看到任何错误提示。

Q：如何修改统计服务的端口？ 打开 `stats_server.py`，修改顶部的 `PORT = 8765` 为其他端口，同时更新 `DDD_system.html` 中 `INTRANET_API` 的端口号。

六、用户使用方法

①顶栏徽章

顶栏右侧原来的"WHO DDD 标准"换成了一个三格计数徽章，页面加载约 1.5 秒后静默拉取数据填入：



点击徽章弹出详情面板。

②统计详情面板（底部上滑）

点击徽章后从底部滑出，包含：

- **环境标签：**自动显示"🏠 内网统计服务"或"🌐 Cloudflare Workers 统计"
- **三张数据卡：**大号数字 + 图标 + 标签，对应三项统计
- **最近访问记录列表：**彩色圆点区分事件类型（蓝=打开、绿=安装、橙=计算），右侧显示时间
- **刷新按钮：**手动重新拉取最新数据

点击面板外区域或右上角 × 关闭。